Object Oriented Programming Using C++

Ketan Kore

Ketan.Kore@sunbeaminfo.com

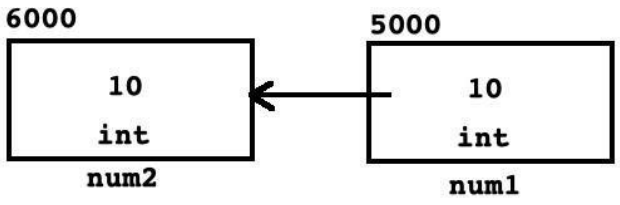
Shallow Copy

- Process of copying contents of object into another object as it is, is called shallow copy.
- Shallow copy is also called as bitwise copy / bit-by-bit copy.
- Compiler by default creates shallow copy.

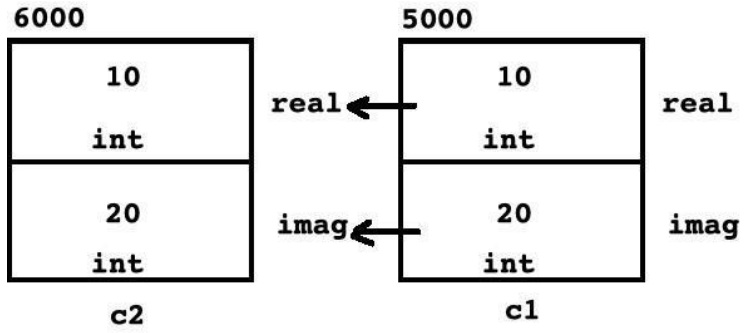
```
int num1 = 10;
int num2 = num1;     //Shallow Copy

Complex c1( 10, 20 );
Complex c2 = c1;        //Shallow Copy
```

Shallow Copy

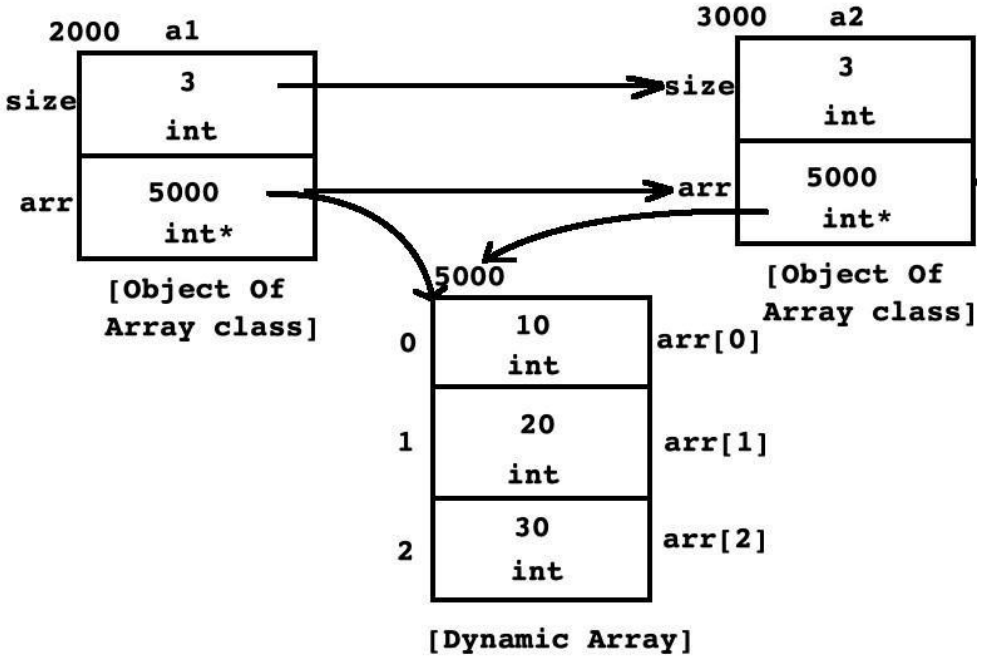



```
int num1 = 10;
int num2 = num1;  //Shallow Copy
```



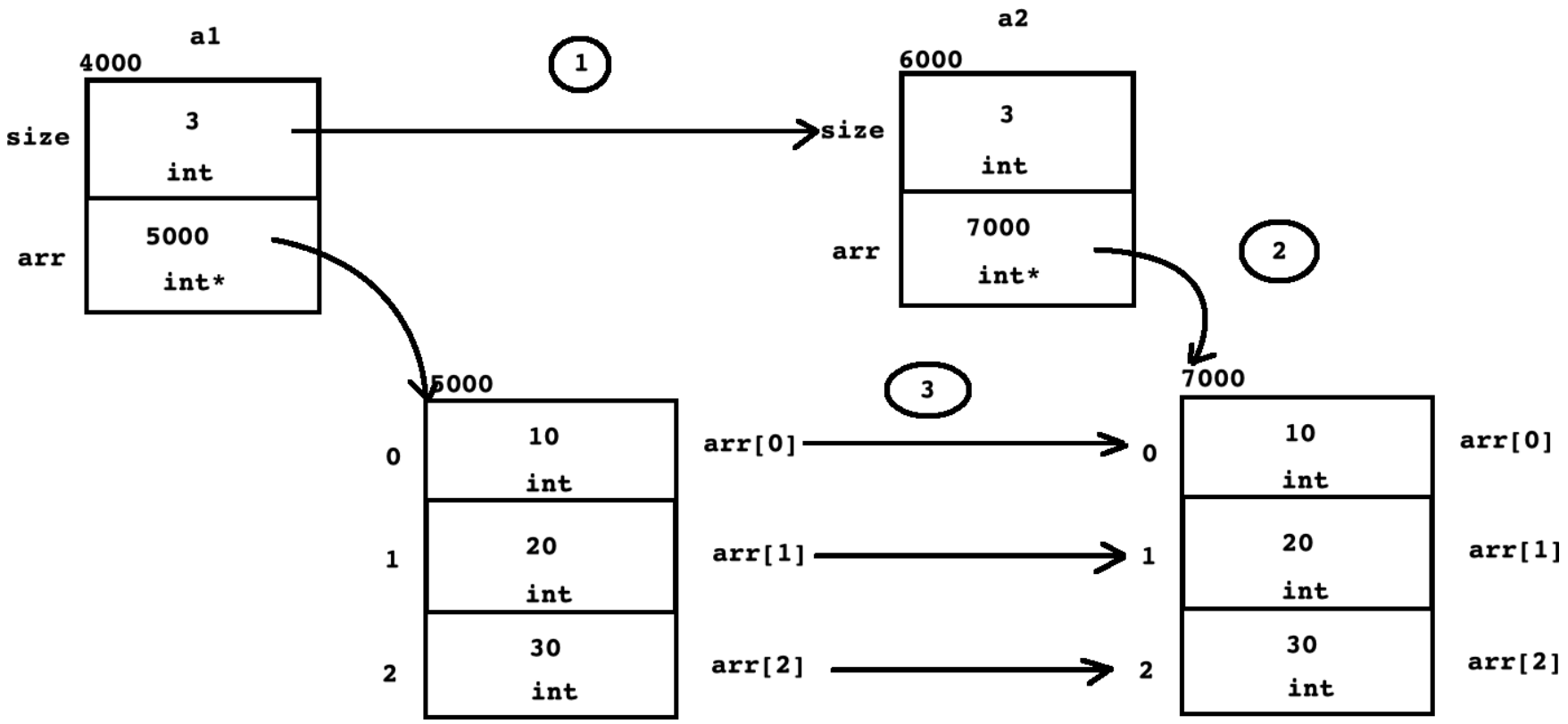

```
Complex c1( 10, 20 );
Complex c2 = c1;  //Shallow Copy
```

```
Array a1( 3 );
a1.acceptRecord( );
Array a2 = a1;
a2.printRecord( );
```



Shallow Copy / Bitwise / Bit-By-Bit Copy

Deep Copy



Deep Copy / Memberwise Copy

Deep Copy

- **Conditions to create deep copy**
    1. Class must contain at least one pointer type data member.
    2. Class must contain user defined destructor.
    3. We must create copy of the object.
- **Steps to create deep copy**
    1. Copy the required size(length, row/col/ array size etc) from source object into destination object.
    2. Allocate new resource for destination object.
    3. Copy the contents from resource of source object into resource of destination object.
- **Location to create deep copy**
    1. In case of assignment, we should create deep copy inside assignment operator function.
    2. In rest of the conditions, we should create deep copy inside copy constructor.

Copy Constructor

- It is parameterized constructor of a class which take single parameter of same type as a reference.
- Job of copy constructor is to initialize object from existing object.

Syntax	Example

`//ClassName &other = source object;`
`//ClassName *const this = address of destination object;`
`ClassName( const ClassName &other ){`
`  //TODO : Shallow / Deep Copy`
`}`
	`//Complex &other = c1;`
	`//Complex *const this = &c2;`
	`Complex( const Complex &other ){`
	`  //Shallow Copy`
	`  this->real = other.real;`
	`  this->imag = other.imag;`
	`}`

- Since copy constructor take single parameter it is considered as parameterized constructor.

Copy Constructor

- If we do not define copy constructor inside class then compiler generates default copy constructor for the class. By default it creates shallow copy.

- **Copy constructor gets called in five conditions**
    1. If we pass object as a argument to the function by value then its copy gets created in function parameter. On function parameter copy constructor gets called.
    2. If we return object from function by value then its copy gets created inside in memory(anonymous object). On anonymous object, copy constructor gets called.
    3. If we initialize object from another object of same class then on newly created object copy constructor gets called.
    4. If we throw object then its copy gets created on environmental stack. Compiler invoke copy constructor on object created on stack.
    5. If we catch object by value then on catching object copy constructor gets called.

Operator Overloading

- In C/C++, we can use operator with the variable fundamental type directly.
- In C language, we can not use operator with objects of user defined type directly/indirectly.
- In C++ language, we can not use operator with objects of user defined type(struct/class) directly.
- In C++, if we want to use operator with objects of user defined type then we should overload operator.
- Overloading operator means defining operator function.
- operator is keyword in C++.
- We can define operator function using 2 ways:
    1. Member function
    2. Non member function.
- By defining operator function, it is possible to use operator with the objects of user defined type. This process of giving extension to the meaning of the operator is called operator overloading

Operator Overloading

- operator overloading using member function:

```
class Complex{
    int real, imag;
public:
    Complex operator+( Complex other ){
        Complex result;
        result.real = this->real + other.real;
        result.imag = this->imag + other.imag;
        return result;
    }
}
c3 = c1 + c2;    //c3 = c1.operator+( c2 );
```

Operator Overloading

- operator overloading using non member function:

```
class Complex{
    int real, imag;
public:
    friend Complex operator+( Complex c1, Complex c2 );
}
Complex operator+( Complex c1, Complex c2 ){
    Complex result;
    result.real = c1.real + c2.real;
    result.imag = c1.imag + c2.imag;
    return result;
}
c3 = c1 + c2;    //c3 = operator+( c1, c2 );
```

Limitations Of Operator

- We can not overload following operators using member function as well as non member function:

    1. Dot / Member selection operator(.)
    2. Pointer to member selcetion operator(.*)
    3. Scope resolution operator(::)
    4. Ternary / Conditional operator
    5. sizeof operator
    6. typeid operator
    7. static_cast operator
    8. dynamic_cast operator
    9. const_cast operator
    10. reinterpret_cast operator

Limitations Of Operator Overloading

- We can not overloading following operators using non `member function.`

  1. `Assignment operator( = )`
  2. `Index / subscript operator( [ ] )`
  3. `Call / function call operator[ ( ) ]`
  4. `Arrow operator`

- Using operator overloading, we can change meaning of `the operator.`
- We can not change number of parameters passed to the `operator function`